Testing and Quality Assurance Report

Course Code: CSE-416

Course Title: Software Project Design and Development

Submitted to: Dr. Mahfida Amjad Dipa

Group Members:

Zannatul Nayem (0432320005101063)

Reon Ahmed (0432320005101066)

Section: 7B

Batch: 54

Semester: Autumn 2026

Project: Automated MCQ Exam Portal

Testing focus: backend unit testing, integration testing, boundary/edge-case analysis, defect correction, and coverage measurement using Jest and NestJS testing utilities.

Prepared from the verified local test run completed in November 2026.

1. Executive Summary

The Automated MCQ Exam Portal was evaluated against the Software Testing and Quality Assurance task supplied for CSE-416. The test work focused on the backend because the core business rules, authentication, authorization, exam configuration, evaluation logic, and controller-service interactions are implemented there. The final verified run completed 42 tests across 9 test suites with zero failures.

42	42	0	24.02%
Tests executed	Tests passed	Failed tests	Statement coverage

Metric	Verified result
Test suites	9 passed / 9 total
Tests	42 passed / 42 total
Statements	24.02%
Branches	20.93%
Functions	16.45%
Lines	23.64%

Overall finding: The final regression suite is green and the core tested areas behave as expected. Overall code coverage remains a baseline because the coverage configuration measures the complete backend, including modules with no dedicated tests yet.

2. Task Alignment and Test Objectives

The instructor task asks the team to: (1) design unit tests using equivalence partitioning and boundary values, (2) write integration tests for key end-to-end flows, (3) run the suite and record pass/fail and coverage, (4) refine edge cases with a test-generation/refinement tool, (5) identify untested defect categories, and (6) fix revealed defects and rerun until the suite passes.

Task step	How this report addresses it
Step 1 - Attempt	Unit tests cover authentication DTO validation, RBAC, exam boundaries, evaluation rules, and existing service tests.
Step 2 - Refine	Additional edge cases were added for invalid partitions, boundary values, grade thresholds, unanswered questions, and role mismatches.
Step 3 - Deepen	Security/negative cases and integration boundaries were added where practical.
Step 4 - Verify	The suite was rerun after correcting test-fixture and test-file defects until 42/42 tests passed.
Deliverables	Test suite, coverage output, test-case matrix, defect/fix log, and this report.

3. System Under Test and Test Strategy

System: Automated MCQ Exam Portal

Primary language: TypeScript

Backend stack: NestJS, Prisma, PostgreSQL/Supabase

Testing stack: Jest, ts-jest, NestJS TestingModule utilities

Frontend stack: React + Vite (outside the verified automated coverage scope in this run)

Unit testing

Individual services, DTO validation rules, role guards, evaluation logic, and exam business rules.

Integration testing

Controller-to-service flows for teacher exam creation and student available-exam lookup.

Equivalence partitioning

Valid and invalid emails, password partitions, valid/invalid roles, and acceptable/rejectable exam settings.

Boundary value analysis

Password length 7/8, passing mark 0/100 and outside range, duration 0/1, question-pool equality, and 49.999/50 grading thresholds.

Negative/security testing

Missing user, mismatched roles, invalid input, ownership/business-rule violations, and invalid exam configuration.

AAA pattern

Tests use Arrange, Act, Assert structure where the behavior under test is isolated.

4. Test Suite Inventory

Test file	Purpose
src/auth/auth.service.spec.ts	Authentication behavior and security-related service cases.
src/evaluation/evaluation.service.spec.ts	Core automatic evaluation behavior.
src/exams/exams.service.spec.ts	Exam service behavior and business rules.
src/questions/questions.service.spec.ts	Question service behavior and option rules.
src/qa/auth-validation.spec.ts	Email/password DTO equivalence and boundary validation.
src/qa/roles.guard.spec.ts	Role-based authorization and missing-user cases.
src/qa/exam-boundaries.spec.ts	Passing mark, duration, and question-pool boundary cases.
src/qa/evaluation-boundaries.spec.ts	Grade thresholds, pass/fail boundaries, and unanswered questions.
src/qa/exams.controller.integration.spec.ts	Controller-to-service integration for create and available-exam flows.

5. Test Case Report

The table below records representative cases from the final suite. All listed cases passed in the verified run. The complete suite contains 42 executable tests.

ID	Scenario	Preconditions / Input	Expected result	Actual	Status
TC-AUTH-01	Valid login DTO	Valid email + password	Validation accepts input	Accepted	PASS
TC-AUTH-02	Invalid email partition	not-an-email	Validation rejects email	Rejected	PASS
TC-AUTH-03	Password lower boundary	8 characters	Validation accepts password	Accepted	PASS
TC-AUTH-04	Password below boundary	7 characters	Validation rejects password	Rejected	PASS
TC-RBAC-01	Matching role	Teacher -> Teacher route	Access granted	Granted	PASS
TC-RBAC-02	Mismatched role	Student -> Admin route	Access denied	Denied	PASS
TC-RBAC-03	Missing user	No authenticated user	Access denied	Denied	PASS
TC-EXAM-01	Passing mark lower boundary	0	Validation accepts	Accepted	PASS
TC-EXAM-02	Passing mark upper boundary	100	Validation accepts	Accepted	PASS
TC-EXAM-03	Duration lower boundary	0 minutes	Validation rejects	Rejected	PASS
TC-EXAM-04	Question pool equality	questionsPerAttempt = pool size	Exam can be created	Created	PASS
TC-EXAM-05	Question pool overflow	questionsPerAttempt = pool + 1	BadRequestException	Rejected	PASS
TC-EVAL-01	Exact pass boundary	50% score	Passed; grade D	Passed	PASS
TC-EVAL-02	Just below pass	49.999% score	Failed; grade F	Failed	PASS
TC-EVAL-03	Grade thresholds	90/80/70/60/50/49.999	A+/A/B/C/D/F	Matched	PASS
TC-EVAL-04	Unanswered question	One answer omitted	Unanswered counted; zero marks	Matched	PASS
TC-INT-01	Teacher create exam	Valid course + questions	Controller delegates and persists	Passed	PASS
TC-INT-02	Student available exams	Valid student + courses	Available exams returned	Passed	PASS

6. Unit Testing Details

Authentication and validation

Covered valid login input, invalid email partition, and registration password boundaries. These cases verify that input validation rejects malformed data while accepting valid boundary values.

Role-based access control

Covered matching roles, mismatched roles, multiple allowed roles, and missing authenticated user. These tests reduce privilege-escalation risk at the guard boundary.

Exam rules

Covered passing-mark limits, zero duration rejection, and exact/overflow question-pool boundaries. These cases target configuration errors before persistence.

Automatic evaluation

Covered exact passing boundary, just-below-pass behavior, representative grade thresholds, negative marking, and unanswered-question counting.

7. Integration Testing Details

Flow	Integration boundary	Expected behavior	Result
Teacher creates exam	ExamsController -> ExamsService -> Prisma mock	Valid exam is persisted with creator and question pool	PASS
Student views available exams	ExamsController -> CoursesService + ExamsService -> Prisma mock	Student course IDs are used to query available exams	PASS

Note: these are service-boundary integration tests, not full browser-driven end-to-end tests against a live production database.

8. Tool-Assisted Refinement Note

The test suite was iteratively refined with AI-assisted review and manual verification of expected values. The instructor task specifically names Qodo (CodiumAI) and Diffblue as example refinement tools. No separate Qodo/Diffblue execution artifact was captured in the verified run, so this report does not claim tool-specific use. If the instructor requires evidence of one of those tools, attach the corresponding screenshot or activity log before submission.

9. Defects / Test Issues Found and Fixed

The final passing run required correction of several test-side defects. These were fixture or test-file problems, not confirmed production application defects.

ID	Observed issue	Root cause	Fix	Final result
QA-01	Evaluation tests counted valid answers as unanswered.	Fixtures omitted selectedOptionId, while the implementation checks it.	Added selectedOptionId to valid answer fixtures.	Resolved; regression passes.
QA-02	Grade-boundary tests produced incorrect expected grades.	The generated score model did not match the service percentage calculation.	Rebuilt boundary data to represent exact 90/80/70/60/50/49.999 percentages.	Resolved; all grade cases pass.
QA-03	TypeScript compiler reported trailing commas / missing brace.	Manual edits left malformed syntax in the new QA spec.	Replaced the affected test block and closed the outer describe block correctly.	Resolved; suite compiles and passes.

Production defect status: No production application defect was confirmed by the final 42-test run. The issues above were test fixture / test-file defects caught and corrected during QA execution.

10. Coverage Report

Coverage was generated with Jest using the project configuration that collects coverage across backend TypeScript sources. The screenshot below is the verified report produced from the 42/42 passing execution.

Metric	Coverage	Covered / Total
Statements	24.02%	210 / 874
Branches	20.93%	45 / 215
Functions	16.45%	26 / 158
Lines	23.64%	175 / 740

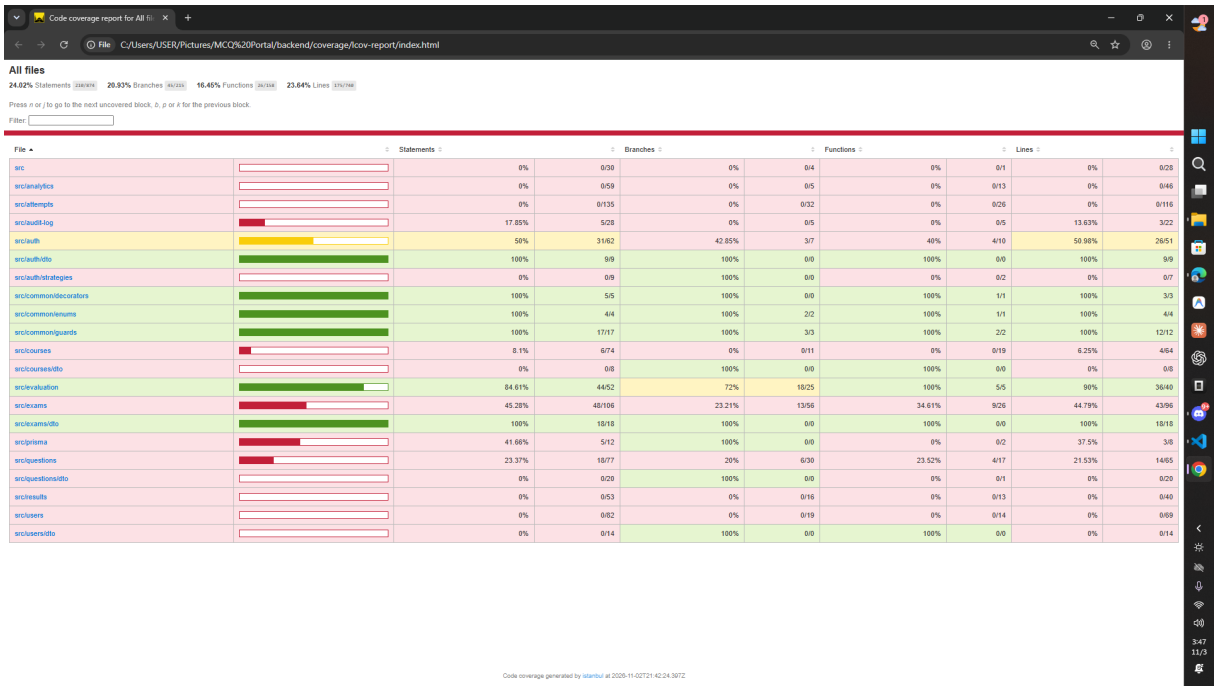


Figure 1. Verified Jest/istanbul coverage report.

Coverage interpretation: core evaluation logic is comparatively well exercised (84.61% statements in the evaluation module shown in the report). Authentication is at 50% statements, exams at 45.28%, questions at 23.37%, while several modules such as results and users remain at 0% in this run. These uncovered areas are the primary candidates for the next test iteration.

11. Test Execution Evidence

Final local execution was performed from the backend directory after test-fixture corrections.

Command	Purpose	Verified outcome
npm test -- --runInBand	Execute complete Jest test suite sequentially.	9 suites passed; 42 tests passed.
npm run test:coverage -- --runInBand	Execute suite and generate coverage report.	Coverage HTML generated; metrics recorded in this report.

GitHub repository

https://github.com/Nayeem974/Automated-MCQ-Exam-Portal

The repository contains the QA test source files and the testing report. Generated coverage output is ignored from Git tracking using the project .gitignore.

12. Recommended Next Coverage Targets

- Add unit tests for attempts, results, users, courses, analytics, and audit-log services.
- Add controller tests for more student, teacher, and admin routes.
- Add a full E2E path using a test database: login -> start exam -> answer -> submit -> evaluate -> result.
- Repeat coverage after the next test iteration and focus on high-risk branches rather than only increasing percentages.

13. Conclusion

The Automated MCQ Exam Portal completed a clean final QA execution with 42 passing tests across 9 suites. The test set covers authentication validation, RBAC, exam configuration boundaries, automatic evaluation, grade thresholds, unanswered questions, and two controller-to-service integration flows. The measured coverage is 24.02% statements, 20.93% branches, 16.45% functions, and 23.64% lines across the complete backend. The report therefore provides a reproducible baseline and a clear list of areas for deeper testing in the next iteration.

Final QA status: PASS
9/9 suites passed | 42/42 tests passed | 0 failed
Coverage report generated and recorded.

Appendix A. Submission Checklist

- Testing & QA PDF report - this document.
- GitHub repository containing unit/integration test source files.
- Coverage HTML report generated locally from Jest.
- Evidence screenshot of final test execution (42/42 passed).
- Evidence screenshot of the coverage summary.
- If required by the instructor, evidence of Qodo/CodiumAI or Diffblue refinement should be attached separately.

Appendix B. Final Test Suite

- src/auth/auth.service.spec.ts
- src/evaluation/evaluation.service.spec.ts
- src/exams/exams.service.spec.ts
- src/questions/questions.service.spec.ts
- src/qa/auth-validation.spec.ts
- src/qa/roles.guard.spec.ts
- src/qa/exam-boundaries.spec.ts
- src/qa/evaluation-boundaries.spec.ts
- src/qa/exams.controller.integration.spec.ts